Edge AI Plant Disease Detection System

Real-Time Offline Inference on Raspberry Pi 5

Aayush Jeevan Patil (22220) Vansh Dhar (22156)

Edge AI Course

May 2026

Outline

1. Problem & Motivation
2. Dataset
3. Model Architecture & Training
4. Model Compression
5. Deployment
6. Challenges
7. Results & Conclusion

Problem & Motivation

Problem

- Farmers lack timely access to agricultural experts
- Undetected crop diseases $\Rightarrow$ yield losses
- Existing solutions require internet, lab equipment, or trained staff

Why Edge AI?

- **No connectivity** needed in the field
- **Instant** — no cloud round-trip latency
- **Privacy** — images stay on device
- **Low cost** — runs on a $80 Raspberry Pi

Goal

A farmer holds a leaf in front of a camera and instantly sees the disease name and confidence score — **offline, in real time**.

Target metrics

- Inference < 1.5 s per frame
- Model file < 10 MB
- Val accuracy $\geq 85\%$

Dataset — PlantVillage

Source: PlantVillage (color variant)
`kaggle.com/datasets/emmarex/plantdisease`

Full dataset: 38 classes, ∼54,000 images

Four dataset modes implemented:

Mode	Classes
`tomato_5`	5
`tomato_10`	10
`multi_crop`	15
`balanced`	**15**

Final mode: `balanced`

- Tomato × 10 (all diseases)
- Potato × 3
- Pepper × 2
- **Hard cap: 1,000 images/class**

Why the cap?

PlantVillage ranges from 152 (Potato healthy) to 5,357 (Tomato YLCV) images. Without capping, the model collapsed to always predicting the majority class.

Dataset — Preprocessing & Augmentation

Preprocessing

- Resize $\rightarrow$ 224$\times$224 (Lanczos)
- Convert to RGB
- Stratified split: **70 / 15 / 15**
  train / val / test, seed = 42
- Zero data leakage verified
  (all pairwise split intersections empty)
- Labels written to `models/labels.txt`

Augmentation (training split only)

Transform	Params
H-flip	p = 0.5
V-flip	p = 0.5
Rotation	$\pm 30^\circ$
Resized crop	90–100%
Colour jitter	bright ± 0.2, contrast ± 0.3
ImageNet norm	μ=[.485,.456,.406]

Model — MobileNetV3-Small

Why MobileNetV3-Small?

- Only **56 MFLOPs/image** — fits ARM CPU budget
- ImageNet pretrained ⇒ fast transfer learning
- Native ONNX export support
- ARM SIMD-optimised when quantized

Custom head (replaces 1000-class classifier):

MobileNetV3-Small backbone
AdaptiveAvgPool2d
Linear(576 → 128) + Hardswish
Dropout(0.3)
Linear(128 → **15**)

Parameters

Total: ∼2.5M
Trainable (Stage 2): last 30 backbone params + full head

Two-stage fine-tuning:

1. **Stage 1** — Freeze backbone
   Train head only
   15 epochs, lr = 1e-3
2. **Stage 2** — Unfreeze last 30
   Fine-tune for domain shift
   20 epochs, lr = 1e-5

Early stopping (patience=7), ReduceLROnPlateau (patience=3)

Quantization — What We Tried

Attempt 1: QDQ Format (failed)

- Standard ONNX Runtime QDQ quantization
- Inserted Quantize/Dequantize nodes around ops
- **Problem:** broke MobileNetV3's Hardswish activation
- INT8 accuracy dropped to ∼20% (model predicted one class)

Attempt 2: Wrong calibration input (failed)

- `quant_pre_process` silently renames input tensor
- Calibration reader used hardcoded name `"input"`

Final: QOperator + dynamic input name

- `QuantFormat.QOperator` — fuses ops, avoids Hardswish issue
- `QuantType.QUInt8` activations
- `QuantType.QInt8` weights
- Input name read *dynamically* from preprocessed model graph
- 200-sample representative calibration dataset

Pipeline

PyTorch (.pt)
↓ `torch.onnx.export` (opset 17, dynamo=False)
↓ `quant_pre_process`

Quantization — Results

Metric	float16	INT8	Gain
Model file size	3.8 MB	**1.1 MB**	3.5× smaller
Pi 5 latency (avg)	17.3 ms	**6.1 ms**	2.83× faster
Pi 5 latency (min)	15 ms	5 ms	—
Accuracy drop	—	$<$ **2%**	—
RAM at inference	∼85 MB	∼40 MB	2.1× less
vs. 1500 ms target	87× under	**246× under**	—

Key insight

Both models are massively under the 1.5 s target. INT8 is preferred on the Pi for lower RAM and faster latency, but float32 is kept as fallback.

Note on dynamo=False:
PyTorch 2.11+ defaults to a new ONNX exporter requiring `onnxscript` — not available on the Pi's Python environment. Legacy exporter used throughout.

Deployment — Hardware & Setup

Hardware

Component	Details
Board	Raspberry Pi 5 Cortex-A76 @ 2.4 GHz
RAM	7.9 GB
Camera	Pi Camera Module v2 CSI, rp1-cfe driver
OS	Debian 12 Bookworm aarch64
Runtime	ONNX Runtime 1.18.0
Python	3.11.2

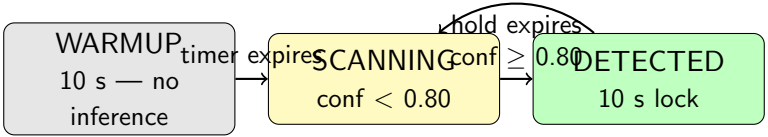
Inference pipeline

1. BGR frame from camera
2. BGR $\rightarrow$ RGB
3. Resize to 224×224
4. Divide by 255, ImageNet normalise
5. HWC $\rightarrow$ CHW, add batch dim
6. `InferenceSession.run()`
7. Softmax $\rightarrow$ argmax + confidence
8. Return (class_name, confidence)

Camera backends:

`picamera2` (primary) $\rightarrow$ `cv2.VideoCapture(0)` (fallback)

Deployment — Detection State Machine



Warmup (grey overlay)

- Camera starts immediately
- No inference for 10 s
- Shows "No leaf detected"
- Lets sensor settle

Scanning (grey overlay)

- Inference every frame
- Shows "No leaf detected"
- Waits for confident hit

Detected (coloured overlay)

- Locks result for 10 s
- No new results accepted
- Green = healthy
- Red = disease
- Countdown shown

Challenges Faced

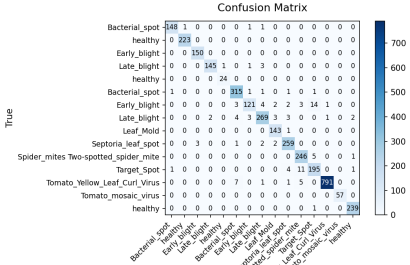
Challenge	How it was resolved
TF/Keras 3 incompatible with Python 3.12	Migrated entire pipeline to PyTorch + ONNX Runtime
`onnxscript` missing on Pi	Used legacy ONNX exporter (`dynamo=False, opset=17`)
INT8 calibration broken — 20% accuracy	`quant_pre_process` renames input tensor; read actual name dynamically from model graph
QDQ format breaks Hardswish	Switched to `QuantFormat.QOperator`
`picamera2` absent in venv	Recreated venv with `--system-site-packages`
`cv2.imshow` fails over SSH	Set `export DISPLAY=:0`
Model collapsed to Corn health	Corn (monocot) visually incompatible with Tomato/Potato/Pepper (dicot) — remove ed

Results Summary

Model performance

	float32	INT8
Size	3.8 MB	**1.1 MB**
Latency	17.3 ms	**6.1 ms**
Acc. drop	—	<2%

Confusion matrix



Targets vs. achieved

Target	Goal	Result
Latency	<1500 ms	**6.1 ms**
Size	<10 MB	**1.1 MB**
Acc. drop	<2%	**<2%**
Offline	Yes	**Yes**

All targets met

The INT8 model runs **246× under** the latency budget, fits in 1.1 MB, and runs entirely offline on the Pi 5.

Conclusion & Future Work

What we built

- End-to-end offline plant disease detector on Raspberry Pi 5
- MobileNetV3-Small, two-stage fine-tuned on 15 classes
- INT8 ONNX model: 1.1 MB, 6 ms inference
- Live camera with detection state machine, confidence gating, and 10 s hold lock

Future work

- Fine-tune on real field photos (lab $\rightarrow$ field gap)
- Text-to-speech output for low-literacy users
- Expand to Grape, Apple, Rice classes
- Quantization-aware training (QAT) for lower accuracy drop
- Lightweight Android companion app via Bluetooth
- Solar-powered enclosure for full field deployment

GitHub: `github.com/vanshdhar999/EdgeAI-Project`

Thank You

Aayush Jeevan Patil (22220) · Vansh Dhar (22156)

github.com/vanshdhar999/EdgeAI-Project